Создание плагинов DITA Studio

Содержание

- 1 Что такое плагин
- 2 Как редактор загружает плагины
- 3 Первый плагин
  - 3.1 Создаём проект плагина
  - 3.2 Устанавливаем и проверяем плагин
- 4 Три вида плагинов
  - 4.1 Правило проверки
  - 4.2 Формат публикации
  - 4.3 Команда режима «Автор»
- 5 Разработка и выпуск
  - 5.1 Отладка и тесты
  - 5.2 Распространение и обновление
- 6 Справочник API
  - 6.1 Документ и элементы
  - 6.2 Правила проверки: интерфейс и замечания
  - 6.3 Форматы публикации: проект, карта, условия
  - 6.4 Документ в «Авторе»: IDocumentView
  - 6.5 Шаблоны файлов
- 7 Если плагин не работает
  - 7.1 Плагина нет в списке
  - 7.2 Сборка не загрузилась
  - 7.3 Плагин падает при работе

1 Что такое плагин

Плагин — библиотека .NET, которая добавляет в DITA Studio правила проверки, форматы публикации или команды режима «Автор». Редактор подхватывает её при запуске, исходный код редактора менять не нужно.

Плагин нужен, когда в команде есть свои правила, которых нет во встроенных: словарь стиля, требования заказчика к структуре, свой формат выдачи (вики, база знаний, система поддержки), повторяющаяся правка текста. Плагин пишется на C# (или любом языке .NET), собирается в `.dll` и кладётся в папку `plugins` рядом с программой.

Таблица 1. Три вида плагинов

Вид и интерфейс	Где появляется	Что получает и что делает
Правило проверки `IValidationRulePlugin`	Правка → Проверить проект (F7) и «Проверить документ» — замечания на вкладке **Проверка**	Документ (`DitaDocument`); возвращает список замечаний с уровнем, текстом и элементом
Формат публикации `IPublishFormatPlugin`	Публикация → Публикация плагином…	Проект, карту, условия сборки и путь к файлу результата; записывает результат
Команда «Автора» `IAuthorCommandPlugin`	Структура → Команда плагина…	Открытый документ (`IDocumentView`): текст, курсор, отмена; меняет документ

Одна `.dll` может содержать сколько угодно плагинов любых видов: каждый — отдельный открытый класс. Если плагинов одного вида несколько, редактор показывает список и предлагает выбрать.

1.1 Что нужно знать заранее

- **.NET 8 и C#.** Плагин — библиотека классов `net8.0`. Подойдёт Visual Studio, Rider, VS Code или просто `dotnet` из командной строки.
- **Немного DITA.** Плагин работает с деревом элементов документа: `topic`, `p`, `step`… Хватит знаний из руководства пользователя.
- **Загрузка — при запуске.** Плагины читаются один раз, когда запускается редактор. Новый или пересобранный плагин начинает работать после перезапуска.

1.2 Безопасность

Плагин — обычный код, он выполняется с правами пользователя: может читать и писать файлы, обращаться к сети. Ставьте только плагины, которым доверяете, лучше — собранные из исходников вашей команды. Ошибка внутри плагина не закрывает редактор: она превращается в сообщение или замечание проверки.

1.3 Пример к руководству

Все примеры собраны в проекте `samples/plugins/DitaStudio.SamplePlugins` репозитория: два правила проверки, экспорт в Markdown и команда «Типографские кавычки». Его можно собрать, установить и использовать как заготовку своего плагина.

2 Как редактор загружает плагины

Где редактор ищет сборки, какие классы считает плагинами и почему ссылки на сборки редактора не копируются в вывод плагина.

При запуске DITA Studio просматривает папку `plugins` рядом со своим исполняемым файлом (`DitaStudio.Desktop.exe` в Windows, `DitaStudio.Desktop` в Linux и macOS). Папка создаётся при первом запуске.

2.1 Какие файлы считаются плагинами

- каждая `.dll` прямо в папке `plugins`;
- в каждой подпапке — одна сборка с именем подпапки: `plugins/MyRules/MyRules.dll`. Остальные `.dll` подпапки — зависимости плагина (например, библиотеки NuGet), плагинами они не считаются.

Подпапка — рекомендуемый способ: всё, что относится к плагину, лежит вместе, удаляется одной папкой, и библиотеки разных плагинов не путаются.

```
plugins/
  DitaStudio.SamplePlugins/
    DitaStudio.SamplePlugins.dll       плагин
  CompanyRules/
    CompanyRules.dll                   плагин
    CompanyRules.deps.json             список зависимостей
    YamlDotNet.dll                     зависимость, не плагин
  QuickFix.dll                         простой плагин без зависимостей
```

2.2 Какие классы становятся плагинами

В каждой сборке редактор ищет классы, которые:

- открытые (`public`) и не абстрактные;
- реализуют один из интерфейсов плагинов;
- имеют открытый конструктор без параметров — редактор создаёт экземпляр сам.

Сборка просматривается трижды — по разу на каждый вид. Класс, реализующий два интерфейса, будет создан дважды, по экземпляру на вид; храните общее состояние в статических полях, если оно нужно.

2.3 Свой контекст загрузки и общие интерфейсы

Каждая сборка плагина грузится в свой `AssemblyLoadContext`: у разных плагинов могут быть разные версии одной и той же библиотеки. Но интерфейсы плагинов (`DitaStudio.Core.dll`, `DitaStudio.Presentation.dll`) должны быть **те же самые**, что у редактора, иначе

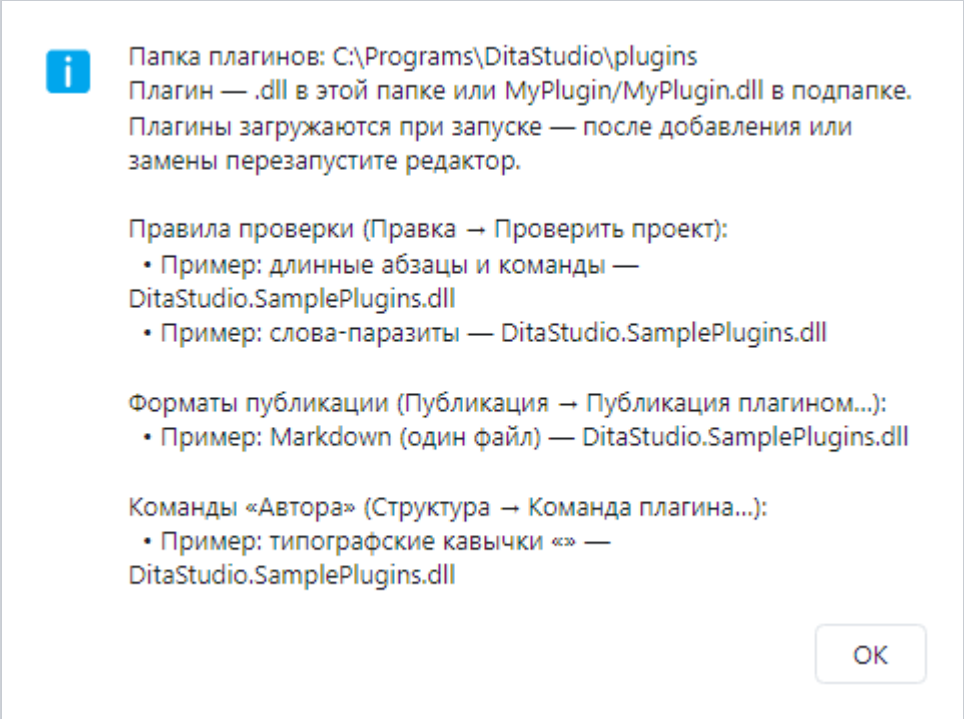
`IValidationRulePlugin` плагина и редактора окажутся двумя разными типами с одинаковым именем, и плагин не найдётся.

Поэтому в проекте плагина ссылки на сборки редактора ставятся с `Private=false` — они не копируются в вывод плагина, и при загрузке плагин получает уже загруженные сборки редактора. Это главное правило; почти все случаи «плагин не появился» — его нарушение (см. Плагина нет в списке).

2.4 Как проверить, что загрузилось

Справка → Плагины… показывает папку плагинов, загруженные плагины по видам с именами файлов и сборки, которые не загрузились, — с причиной.

Рисунок 1. Окно «Справка → Плагины…»



Если какая-то сборка не загрузилась, при запуске в строке состояния появляется «Плагины: не загрузилось N — см. „Справка → Плагины…“».

2.5 Ошибки во время работы

Исключение из плагина не закрывает редактор:

- правило проверки — предупреждение «Плагин … упал при проверке» на вкладке **Проверка** для каждого файла, на котором это случилось, остальные правила и файлы проверяются;
- формат публикации и команда «Автора» — окно с текстом исключения, в журнале сборки — строка ошибки.

3 Первый плагин

3.1 Создаём проект плагина

Библиотека классов .NET 8 со ссылками на сборки установленного редактора — без копирования их в вывод плагина.

3.1.1 Перед началом

Установлены .NET 8 SDK (`dotnet --version` показывает 8.x или новее) и DITA Studio — например, в `C:\Programs\DitaStudio`. В папке программы лежат `DitaStudio.Core.dll` и `DitaStudio.Presentation.dll` — на них и ссылается плагин.

Порядок действий

1. **Создайте библиотеку классов.**

   ```
   dotnet new classlib -n CompanyRules -f net8.0
   cd CompanyRules
   ```

   Имя проекта станет именем сборки и папки плагина: `plugins/CompanyRules/CompanyRules.dll`.

2. **Замените содержимое `CompanyRules.csproj`.**

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <!-- В вывод — только сборка плагина. -->
    <CopyLocalLockFileAssemblies>false</CopyLocalLockFileAssemblies>
    <!-- Папка установленного редактора: ссылки на его сборки и установка плагина
после сборки. -->
    <DitaStudioDir>C:\Programs\DitaStudio</DitaStudioDir>
  </PropertyGroup>

  <ItemGroup>
    <!-- Private=false: сборки редактора НЕ копируются в вывод плагина. -->
    <Reference Include="DitaStudio.Core">
      <HintPath>$(DitaStudioDir)\DitaStudio.Core.dll</HintPath>
      <Private>false</Private>
    </Reference>
    <!-- Нужна только командам «Автора» (IAuthorCommandPlugin, IDocumentView). -->
    <Reference Include="DitaStudio.Presentation">
      <HintPath>$(DitaStudioDir)\DitaStudio.Presentation.dll</HintPath>
      <Private>false</Private>
    </Reference>
  </ItemGroup>

  <!-- После сборки — копия в plugins/<Имя>/ установленного редактора (перезапустите
его). -->
  <Target Name="InstallPlugin" AfterTargets="Build"
Condition="Exists('$(DitaStudioDir)')">
    <Copy SourceFiles="$(TargetPath)"
DestinationFolder="$(DitaStudioDir)\plugins\$(AssemblyName)" />
  </Target>

</Project>
```

Что здесь важно:

- `Private=false` у ссылок на сборки редактора — их не должно быть в выводе плагина (см. Как редактор загружает плагины);
- `DitaStudioDir` — папка установленного редактора; поменяйте на свою;
- цель `InstallPlugin` после каждой сборки кладёт плагин в `plugins/CompanyRules/` — остаётся перезапустить редактор.

Ссылка на `DitaStudio.Presentation.dll` нужна только для команд «Автора»; правилам проверки и форматам публикации хватает `DitaStudio.Core.dll`.

3. **Удалите заготовку `Class1.cs` и добавьте класс плагина.**

   Как написать каждый вид — в следующих главах. Для пробы годится правило из главы Правило проверки.

4. **Соберите проект.**

   ```
   dotnet build -c Release
   ```

   →

   В `bin/Release/net8.0` — `CompanyRules.dll` (и `.pdb`, `.deps.json`), но **нет** `DitaStudio.Core.dll`. Если она там есть — ссылка поставлена без `Private=false`.

3.1.2 Результат

Проект готов. В репозитории редактора есть такой же проект с примерами всех трёх видов — `samples/plugins/DitaStudio.SamplePlugins`. В репозитории он ссылается на исходники редактора, а в архиве выпуска (папка `samples/plugins` рядом с программой) — на сборки самой программы и собирается сразу: `dotnet build -c Release`.

3.2 Устанавливаем и проверяем плагин

Плагин копируется в папку plugins рядом с программой, редактор перезапускается, а окно «Справка → Плагины...» показывает, что он загрузился.

Порядок действий

1. **Закройте DITA Studio.**

   Запущенный редактор держит загруженные сборки плагинов открытыми: заменить файл не получится, а новый не подхватится до перезапуска.

2. **Скопируйте сборку плагина в `plugins/<Имя плагина>/` рядом с программой.**

   ```
   mkdir "C:\Programs\DitaStudio\plugins\CompanyRules"
   copy bin\Release\net8.0\CompanyRules.dll
   "C:\Programs\DitaStudio\plugins\CompanyRules\"
   ```

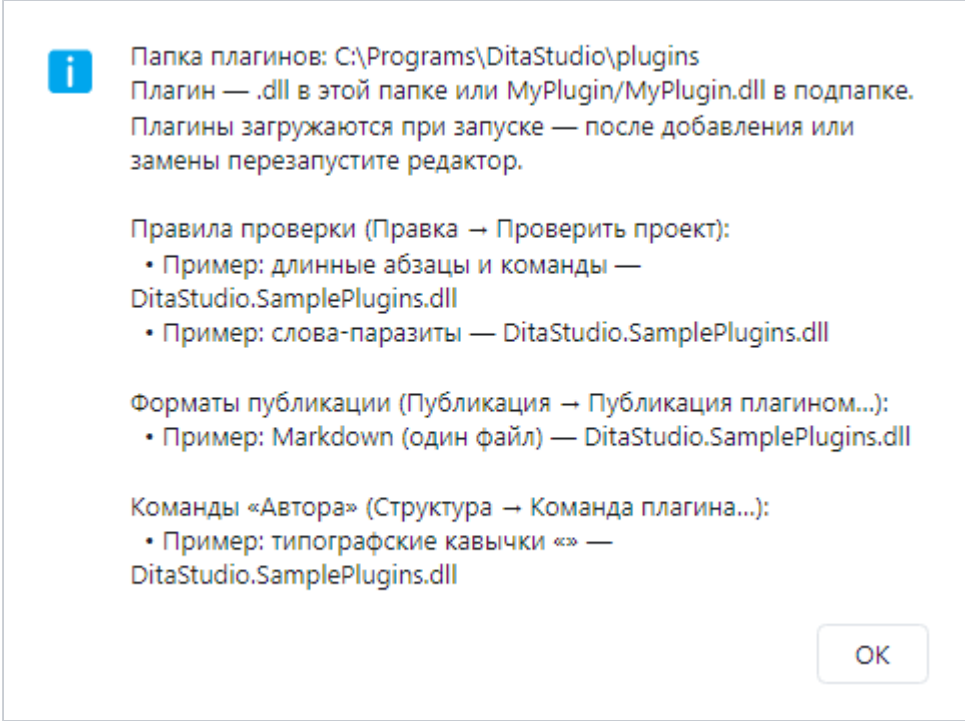
   У плагина есть свои зависимости (библиотеки NuGet)? Скопируйте в ту же подпапку весь вывод сборки, кроме сборок редактора. Цель `InstallPlugin` из шаблона проекта делает копирование сама.

3. **Запустите редактор и откройте Справка → Плагины...**

   →

   Плагин — в списке своего вида, рядом с ним — имя файла сборки.

Рисунок 2. Загруженные плагины



4. **Запустите плагин там, где он появляется.**

Вид	Как запустить
Правило проверки	F7 — замечания плагина на вкладке **Проверка** вместе со встроенными
Формат публикации	выберите карту на вкладке **Карта**, затем Публикация → Публикация плагином...
Команда «Автора»	откройте документ, затем Структура → Команда плагина...

3.2.1 Результат

Чтобы удалить плагин, закройте редактор и удалите его подпапку из `plugins`.

---

4 Три вида плагинов

4.1 Правило проверки

Класс с интерфейсом IValidationRulePlugin получает каждый документ проекта и возвращает замечания — они появляются на вкладке «Проверка» рядом со встроенными.

4.1.1 Контекст

Правило вызывается для каждого файла проекта при Правка → Проверить проект и для открытого документа при «Проверить документ». Правило получает `DitaDocument` — разобранный файл с деревом элементов — и ничего в нём не меняет.

Порядок действий

1. **Создайте открытый класс, реализующий `IValidationRulePlugin`.**

```
using DitaStudio.Core.Model;
using DitaStudio.Core.Validation;

namespace CompanyRules;

public sealed class NoTodoRule : IValidationRulePlugin
{
    public string Name => "Нет пометок TODO";

    public IEnumerable<ValidationIssue> Check(DitaDocument document)
    {
        foreach (var node in document.Root.DescendantsAndSelf())
        {
            if (node.Kind == NodeKind.Text && node.Value.Contains("TODO",
StringComparison.Ordinal))
            {
                yield return new ValidationIssue(IssueSeverity.Error,
                    "В тексте осталась пометка TODO", node.Parent,
document.FilePath);
            }
        }
    }
}
```

   - `Name` — название в окне «Плагины» и в сообщении, если правило упадёт;
   - `Check` — обход дерева документа и замечания через `yield return`.

2. **Выберите уровень замечания.**

Уровень	Когда
`IssueSeverity.Error`	документ нельзя публиковать: битая ссылка, запрещённое содержимое, пометка TODO
`IssueSeverity.Warning`	стоит исправить: нарушение словаря стиля
`IssueSeverity.Info`	рекомендация: длинная фраза, нет иллюстрации

3. **Передайте в замечание элемент и путь к файлу.**

   Третий параметр `ValidationIssue` — узел, к которому переходит двойной щелчок по строке замечания; передавайте элемент (`node.Parent` для текста), а не текстовый узел.

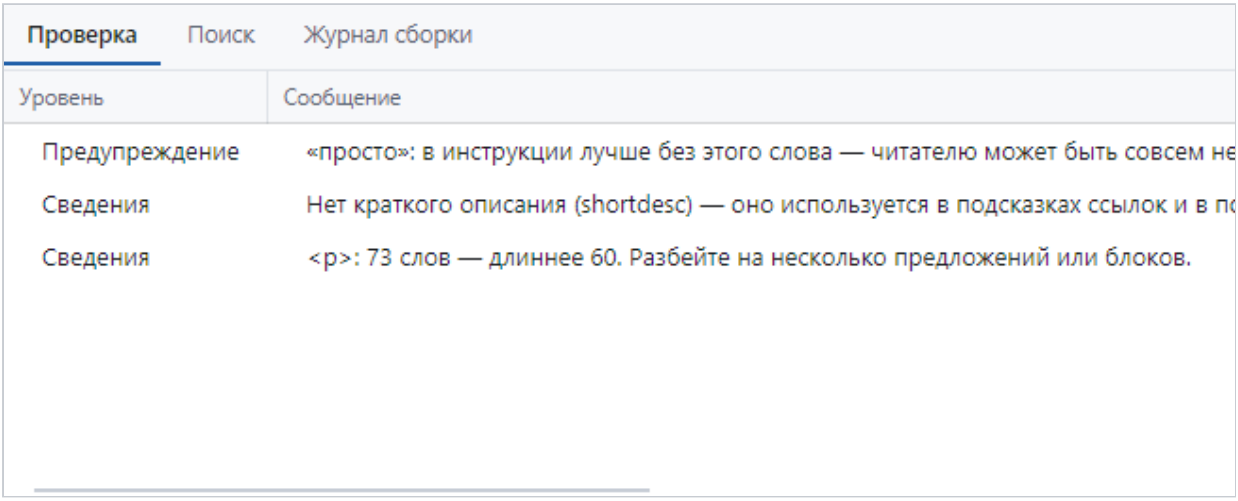
Четвёртый — `document.FilePath`: при проверке всего проекта по нему открывается нужный файл.

4. **Соберите, установите, перезапустите редактор и нажмите F7.**

   →

   Замечания правила — в общем списке. На снимке — два правила из примера: «слова-паразиты» (предупреждение) и «длинные абзацы» (сведения).

Рисунок 3. Замечания правил из плагина на вкладке «Проверка»



4.1.2 Результат

Правило работает со всеми видами топиков и с картами — проверяйте имя корневого элемента (`document.Root.Name`), если правило относится только к задачам или только к картам. Правило видит один файл: связи между файлами (ссылки, ключи) проверяет сам редактор.

4.1.3 Пример: два правила стиля

Файл `StyleRules.cs` примера: «длинные абзацы и команды» (слова считаются по `InnerText`, для команды шага предел строже) и «слова-паразиты» (текст внутри кода не проверяется).

```
using DitaStudio.Core.Model;
using DitaStudio.Core.Validation;

namespace DitaStudio.SamplePlugins;

/// <summary>
/// Правило проверки: слишком длинные абзацы, пункты списков и команды шагов. Длинную
фразу трудно прочитать с первого раза,
/// а команду шага — выполнить. Замечание уровня «Сведения»: рекомендация, а не ошибка.
/// </summary>
public sealed class LongBlockRule : IValidationRulePlugin
{
    /// <summary>Больше скольких слов — уже «длинно».</summary>
    private const int MaxWords = 60;

    /// <summary>Для команды шага предел строже: одно действие — одно короткое
предложение.</summary>
    private const int MaxCommandWords = 25;

    private static readonly HashSet<string> Blocks = new(StringComparer.Ordinal) { "p",
"li", "cmd" };

    public string Name => "Пример: длинные абзацы и команды";

    public IEnumerable<ValidationIssue> Check(DitaDocument document)
    {
        foreach (var node in document.Root.Descendants())
        {
            if (node.Kind != NodeKind.Element || !Blocks.Contains(node.Name))
            {
                continue;
            }

            var words = node.InnerText.Split((char[]?)null,
StringSplitOptions.RemoveEmptyEntries).Length;
            var limit = node.Name == "cmd" ? MaxCommandWords : MaxWords;
            if (words > limit)
            {
                // Узел — для перехода двойным щелчком из вкладки «Проверка», путь к
файлу — для проверки всего проекта.
                yield return new ValidationIssue(IssueSeverity.Info,
                    $"<{node.Name}>: {words} слов — длиннее {limit}. Разбейте на
несколько предложений или блоков.",
                    node, document.FilePath);
            }
        }
    }
}

/// <summary>
/// Правило проверки по словарю стиля: слова, которых в технической документации лучше
избегать («просто», «очевидно»…).
/// Код и имена файлов не проверяются — в них такие слова могут быть частью имени.
```

```
/// </summary>
public sealed class StopWordsRule : IValidationRulePlugin
{
    private static readonly string[] StopWords = { "просто", "очевидно", "легко", "всего
лишь", "и т. д." };

    /// <summary>Элементы, текст которых не проверяется.</summary>
    private static readonly HashSet<string> CodeElements = new(StringComparer.Ordinal)
    {
        "codeph", "codeblock", "filepath", "cmdname", "varname", "parmname", "apiname",
"userinput", "systemoutput", "msgph", "screen"
    };

    public string Name => "Пример: слова-паразиты";

    public IEnumerable<ValidationIssue> Check(DitaDocument document)
    {
        foreach (var text in document.Root.Descendants().Where(n => n.Kind ==
NodeKind.Text))
        {
            if (text.Ancestors().Any(a => CodeElements.Contains(a.Name)))
            {
                continue;
            }

            foreach (var word in StopWords)
            {
                if (ContainsWord(text.Value, word))
                {
                    // Замечание ставится на элемент, в котором стоит текст: к нему
переходит двойной щелчок.
                    yield return new ValidationIssue(IssueSeverity.Warning,
                        $"«{word}»: в инструкции лучше без этого слова — читателю может
быть совсем не просто.",
                        text.Parent, document.FilePath);
                }
            }
        }
    }

    /// <summary>Слово целиком, без учёта регистра: «просто» находится, «простой» — нет.
</summary>
    private static bool ContainsWord(string text, string word)
    {
        for (var at = text.IndexOf(word, StringComparison.OrdinalIgnoreCase); at >= 0;
             at = text.IndexOf(word, at + 1, StringComparison.OrdinalIgnoreCase))
        {
            var before = at == 0 || !char.IsLetter(text[at - 1]);
            var end = at + word.Length;
            var after = end >= text.Length || !char.IsLetter(text[end]);
            if (before && after)
            {
                return true;
            }
```

```
        }

        return false;
    }
}
```

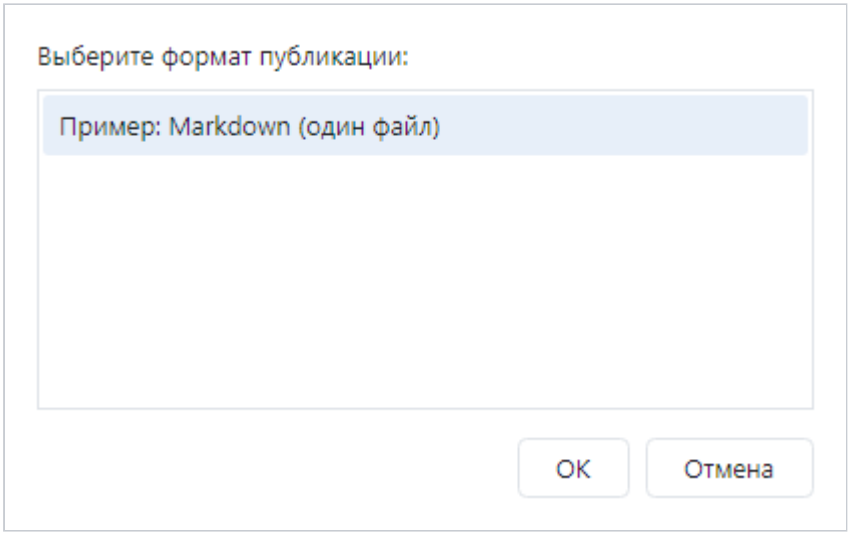
4.2 Формат публикации

Класс с интерфейсом IPublishFormatPlugin получает проект, карту, условия сборки и путь к файлу результата — и записывает публикацию в своём формате.

4.2.1 Контекст

Пользователь выбирает карту на вкладке **Карта**, затем Публикация → Публикация плагином…. Редактор сохраняет открытые документы, пересобирает ключи, спрашивает формат (если плагинов несколько) и имя файла и вызывает `Publish`. Готовый файл он открывает программой по умолчанию.

Рисунок 4. Выбор формата публикации



Порядок действий

1. **Создайте класс, реализующий `IPublishFormatPlugin`.**

```
public sealed class MarkdownFormat : IPublishFormatPlugin
{
    public string Name => "Пример: Markdown (один файл)";

    public string FileExtension => "md";   // без точки: окно сохранения предложит
*.md

    public void Publish(DitaProject project, string mapPath, PublishOptions options,
string outputPath)
    {
        // 1. Дерево карты без строк, исключённых условиями сборки.
        var tree = MapTree.Build(project, mapPath, node =>
PublishFilter.IsIncluded(node, options));
        var sb = new StringBuilder("# " + tree.Root.Title + "\n\n");

        // 2. Топики и разделы без своего топика (topichead) в порядке карты.
        foreach (var item in tree.PublicationOrderWithHeadings)
        {
            var level = Math.Clamp(item.Level + 1, 2, 6);
            if (item.IsHeadingOnly)
            {
                sb.Append(new string('#', level)).Append('
').Append(item.Title).Append("\n\n");
                continue;
            }

            // 3. Документ топика с подставленными conref.
            var document = RefResolver.ExpandConrefs(project,
project.GetDocument(item.TargetPath!));
            // ... перевод элементов в Markdown; элементы с исключёнными условиями
пропускаются:
            //     PublishFilter.IsIncluded(element, options); текст ключей —
project.TextWithKeys(element, item.KeyScopeChain)
        }

        // 4. Запись через временный файл: при сбое прежний результат не портится.
        AtomicFile.WriteAllText(outputPath, sb.ToString(), new UTF8Encoding(false));
    }
}
```

2. **Постройте дерево карты с учётом условий сборки.**

   `PublishOptions` приходит уже с исключениями из «Условий сборки» и подключённого `.ditaval`. `MapTree.Build` с фильтром `PublishFilter.IsIncluded` убирает исключённые строки карты вместе с ветками; внутри топиков тот же фильтр вызывается для каждого элемента.

3. **Обойдите `PublicationOrderWithHeadings`.**

   Это порядок издания: топики (`TargetPath` — файл) и разделы без своего топика (`IsHeadingOnly`, у них только `Title`). `Level` — глубина в карте: 1 — глава, 2 — раздел главы. Строки «только ресурс» и битые ссылки сюда не входят.

4. **Для каждого топика подставьте `conref` и ключи.**

   `RefResolver.ExpandConrefs` возвращает копию документа с подставленными фрагментами; исходный документ не меняется. Текст пустого элемента с ключом (`<keyword keyref="product"/>`) — `project.TextWithKeys(element, item.KeyScopeChain)`.

5. **Запишите результат через `AtomicFile`.**

   `AtomicFile.WriteAllText` пишет во временный файл рядом и заменяет результат, только когда он готов целиком. Картинки и другие файлы рядом с результатом — `Path.GetDirectoryName(outputPath)`.

6. **Сообщайте об ошибках исключением.**

   Текст исключения редактор показывает в окне и пишет в «Журнал сборки». Отдельного списка предупреждений у плагина публикации нет.

4.2.2 Результат

Так выглядит начало результата примера для учебного проекта `samples/GuideSample`:

```
# Руководство пользователя «Пример»

## О продукте

_Пример 2.0 — учебный пример проекта DITA, на котором удобно посмотреть, как устроены концепции, задачи, справка и глоссарий._

### Установка

1. Запустите `setup.exe`.
   Если появится запрос контроля учётных записей, подтвердите его.

3. Выберите вариант установки.
   | Вариант | Когда выбирать |
   | --- | --- |
   | Обычная | Подходит большинству пользователей. |

> **Совет.** Начните с задачи Установка, если хотите сразу попробовать продукт.
```

Полный код — `MarkdownFormat.cs` в проекте примера: заголовки, краткое описание, абзацы, списки и шаги, примечания, блоки кода, рисунки, таблицы всех видов, списки определений и параметров, фразовое оформление и ссылки.

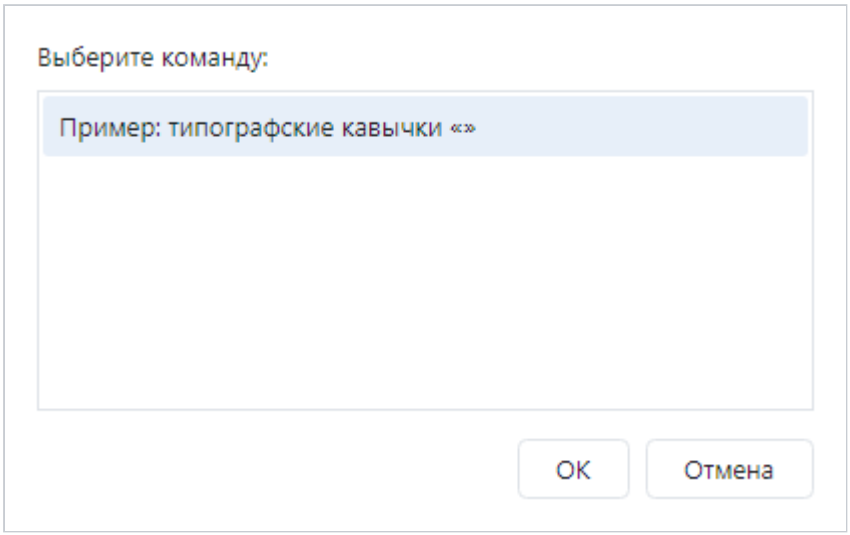
4.3 Команда режима «Автор»

Класс с интерфейсом IAuthorCommandPlugin получает открытый документ и меняет его: правит текст, вставляет элементы, работает с элементом под курсором. Отмена — одним шагом.

4.3.1 Контекст

Пользователь открывает документ и выбирает Структура → Команда плагина…. Редактор переносит в модель недописанный ввод, вызывает `Execute`, а затем сам помечает документ изменённым, перестраивает «Автор» и панель атрибутов и пишет в строке состояния «Команда плагина … выполнена».

Рисунок 5. Выбор команды плагина



Порядок действий

1. **Добавьте в проект ссылку на `DitaStudio.Presentation.dll` (с `Private=false`) и создайте класс, реализующий `IAuthorCommandPlugin`.**

2. **Первым делом в `Execute` вызовите `pane.PushUndo(Name)`.**

   Это снимок документа до правки: пункт «Отменить» (Ctrl+Z) вернёт документ одним шагом. Без снимка правку плагина не отменить.

3. **Меняйте модель: `pane.Document` (всё дерево) или `pane.Author.CurrentNode` (элемент под курсором).**

   Узлы правятся методами `DitaNode`: `Value` текстового узла, `SetAttribute`, `Add`/`Insert`/`RemoveSelf`, `DitaNode.Element("p")` и `DitaNode.Text("…")` для новых узлов. Готовые структурные операции «Автора» — через `pane.Author`: `InsertElement`, `WrapCurrentInline`, `MoveCurrent`, `DeleteCurrent` (см. Документ в «Авторе»).

   > **Важно:** Редактор не проверяет результат по контент-модели DITA, как делает при правке руками. Вставляйте только допустимые элементы и проверяйте результат (F7) — например, в тестах плагина.

4. **Если нужно показать элемент, вызовите `pane.FocusNode(node)`.**

   Курсор встанет в блок с этим элементом; `false` — элемент в «Авторе» не показывается (например, в прологе).

4.3.2 Результат

Пример — команда «Типографские кавычки»: прямые кавычки во всём документе становятся «ёлочками», код не трогается.

```
using System.Text;
using DitaStudio.Core.Model;
using DitaStudio.Presentation.Plugins;
using DitaStudio.Presentation.Services;

namespace DitaStudio.SamplePlugins;

/// <summary>
/// Команда режима «Автор»: прямые кавычки "…" во всём открытом документе заменяются
«ёлочками». Код, имена файлов и команд не трогаются.
/// Показывает главное для команды: снимок для отмены до правки (<see
cref="IDocumentView.PushUndo"/>), правку модели документа и
/// что редактор делает после команды сам (помечает документ изменённым и перестраивает
«Автор»).
/// </summary>
public sealed class TypographicQuotesCommand : IAuthorCommandPlugin
{
    private static readonly HashSet<string> CodeElements = new(StringComparer.Ordinal)
    {
        "codeph", "codeblock", "pre", "screen", "filepath", "cmdname", "varname",
"parmname", "apiname", "userinput", "systemoutput", "msgph"
    };

    public string Name => "Пример: типографские кавычки «»";

    /// <summary>Сколько кавычек заменено при последнем запуске (для тестов и отладки).
</summary>
    public int LastReplaced { get; private set; }

    public void Execute(IDocumentView pane)
    {
        // Снимок ДО правки: «Отменить» (Ctrl+Z) вернёт документ одним шагом.
        pane.PushUndo(Name);

        LastReplaced = 0;
        var previous = ' '; // символ перед кавычкой — по нему видно, открывающая она или
закрывающая
        foreach (var text in pane.Document.Root.Descendants().Where(n => n.Kind ==
NodeKind.Text).ToList())
        {
            if (text.Ancestors().Any(a => CodeElements.Contains(a.Name)))
            {
                previous = 'x';
                continue;
            }

            var sb = new StringBuilder(text.Value.Length);
            foreach (var c in text.Value)
            {
                if (c == '"')
                {
                    // После пробела, начала текста или открывающей скобки — открывающая,
иначе закрывающая.
```

```
                    sb.Append(char.IsWhiteSpace(previous) || previous is '(' or '[' or
'«' ? '«' : '»');
                    LastReplaced++;
                }
                else
                {
                    sb.Append(c);
                }

                previous = c;
            }

            text.Value = sb.ToString();
        }

        // Ничего больше делать не нужно: после Execute редактор помечает документ
изменённым, перестраивает «Автор» и пишет
        // в строке состояния «Команда плагина … выполнена». Сообщить пользователю что-то
своё плагин может исключением —
        // его текст редактор покажет в окне (так делают с ошибками, а не с обычным
итогом).
    }
}
```

5 Разработка и выпуск

5.1 Отладка и тесты

Правила проверки и форматы публикации проверяются обычными модульными тестами без редактора; для отладки в редакторе к нему подключается отладчик.

Порядок действий

1. **Создайте проект тестов xUnit рядом с плагином.**

   ```
   dotnet new xunit -n CompanyRules.Tests -f net8.0
   dotnet add CompanyRules.Tests reference CompanyRules
   ```

   В проекте тестов ссылка на `DitaStudio.Core.dll` — обычная, с копированием: тесты запускаются без редактора, и сборка должна лежать рядом.

```
<ItemGroup>
  <!-- В тестах сборки редактора нужны в выводе: Private по умолчанию (true). -->
  <Reference Include="DitaStudio.Core">
    <HintPath>C:\Programs\DitaStudio\DitaStudio.Core.dll</HintPath>
  </Reference>
  <ProjectReference Include="..\CompanyRules\CompanyRules.csproj" />
</ItemGroup>
```

2. **Проверьте правило на документе из строки и на целом проекте.**

```
using DitaStudio.Core.Model;
using DitaStudio.Core.Project;
using DitaStudio.Core.Validation;
using Xunit;

public class NoTodoRuleTests
{
    [Fact]
    public void FindsTodoInText()
    {
        var doc = DitaDocument.Parse("<topic id=\"t\"><title>Тема</title><body>
<p>TODO: дописать</p></body></topic>");
        var issue = Assert.Single(new NoTodoRule().Check(doc));
        Assert.Equal(IssueSeverity.Error, issue.Severity);
        Assert.Equal("p", issue.Node?.Name);
    }

    [Fact]
    public void WholeProject()
    {
        var project = new DitaProject(@"..\..\..\TestProject");
        project.Scan();
        var issues = project.ValidateAll(new IValidationRulePlugin[] { new
NoTodoRule() });
        Assert.DoesNotContain(issues, i => i.Message.Contains("TODO"));
    }
}
```

   `DitaDocument.Parse` разбирает XML из строки, `DitaProject` + `Scan()` — папку проекта целиком. `ValidateAll(plugins)` — то же, что F7 в редакторе: встроенная проверка плюс ваши правила.

3. **Формат публикации проверяйте так же: `new MyFormat().Publish(project, mapPath, new PublishOptions(), outputPath)` и сравнение результата.**

4. **Чтобы отладить плагин в редакторе, подключите отладчик к процессу `DitaStudio.Desktop`.**

   В Visual Studio — Отладка → Присоединиться к процессу, в Rider — Attach to Process. Точки останова в коде плагина сработают, если рядом со сборкой лежит её `.pdb`.

Другой способ — запуск редактора из среды разработки: в свойствах отладки проекта плагина укажите программу `C:\Programs\DitaStudio\DitaStudio.Desktop.exe`, а сборка с целью `InstallPlugin` сама положит плагин на место. Остановиться в самом начале можно вызовом `System.Diagnostics.Debugger.Launch()` в коде плагина (уберите его перед выпуском).

5.1.1 Результат

Команды «Автора» требуют открытого документа, их удобнее проверять в редакторе. Пример полного набора тестов для всех трёх видов — `tests/DitaStudio.Desktop.Tests/SamplePluginTests.cs` в репозитории редактора.

5.2 Распространение и обновление

Плагин передаётся папкой со сборкой; при обновлении редактора его стоит пересобрать с новыми сборками редактора и прогнать тесты.

Порядок действий

1. **Упакуйте подпапку плагина в архив.**

   В архиве — папка `CompanyRules` с `CompanyRules.dll` и, если есть, зависимостями и `.deps.json`. Пользователь распаковывает её в `plugins` и перезапускает редактор. Сборок редактора в архиве быть не должно.

2. **Укажите в описании плагина версию DITA Studio, для которой он собран.**

   Задайте и версию самой сборки (`<Version>1.2.0</Version>` в `.csproj`) — по ней видно, какая сборка установлена.

3. **После обновления редактора пересоберите плагин с новыми `DitaStudio.Core.dll` и `DitaStudio.Presentation.dll` и прогоните тесты.**

   Интерфейсы плагинов меняются редко и стараются оставаться совместимыми, но классы ядра, которыми пользуется плагин (`MapTree`, `PublishOptions`…), могут меняться между версиями. Ошибка вида `Method not found` или `Could not load type` в окне «Плагины» или при запуске плагина — признак того, что плагин собран для другой версии.

5.2.1 Результат

Для команды удобно хранить плагины в одном репозитории с документацией и собирать их вместе с ней в CI: тесты плагина заодно проверяют проект документации.

6 Справочник API

6.1 Документ и элементы

DitaDocument — разобранный файл DITA, DitaNode — узел его дерева: элемент, текст или комментарий. С ними работают все три вида плагинов.

Таблица 2. DitaDocument (DitaStudio.Core.Model)

Член	Что даёт
`Root`	Корневой элемент: топик (concept, task…) или карта
`FilePath`	Полный путь к файлу; null у документа из строки
`Title`	Заголовок топика или карты (у статьи глоссария — glossterm)
`Id`	id корневого элемента
`Kind`	Вид: топик, карта и т. п. (DitaDocumentKind)
`IsDirty`	Есть несохранённые правки
`DitaDocument.Parse(xml)`	Документ из строки — для тестов
`DitaDocument.Load(path)`	Документ из файла (в редакторе лучше project.GetDocument — с кэшем)
`ToXmlString()`	XML документа

Таблица 3. DitaNode (DitaStudio.Core.Model)

Член	Что даёт
`Kind`	NodeKind.Element, Text, Comment, ProcessingInstruction
`Name`	Имя элемента (p, step, keyword…); у текста — #text
`Value`	Текст текстового узла и комментария (можно менять)
`InnerText`	Весь текст элемента со всеми вложенными (ключи не подставлены)
`GetAttribute(name)`, `HasAttribute`, `SetAttribute(name, value)`, `RemoveAttribute`	Атрибуты; SetAttribute с null удаляет атрибут
`Parent`, `Children`, `ElementChildren()`	Родитель, все дети, только дочерние элементы
`Descendants()`, `DescendantsAndSelf()`, `Ancestors()`	Обход поддерева и цепочки предков
`FirstElement(name)`, `FindDescendant(name)`, `Closest(name)`	Первый дочерний, первый вложенный, ближайший предок (или сам) с таким именем
`PreviousSibling`, `NextSibling`, `IndexInParent`	Соседи и место у родителя
`DitaNode.Element(name)`, `DitaNode.Text(value)`	Новые узлы
`Add`, `Insert(index, node)`, `Remove`, `RemoveSelf()`, `ReplaceWith`, `SetText(text)`	Правка дерева (в командах «Автора»)
`CloneDeep()`	Копия поддерева
`Line`, `Column`	Место в исходном файле

6.2 Правила проверки: интерфейс и замечания

Интерфейс IValidationRulePlugin и класс замечания ValidationIssue.

6.2.1 IValidationRulePlugin (DitaStudio.Core.Validation)

```
public interface IValidationRulePlugin
{
    string Name { get; }
    IEnumerable<ValidationIssue> Check(DitaDocument document);
}
```

Таблица 4. ValidationIssue

Параметр конструктора	Что задаёт
`IssueSeverity severity`	`Error`, `Warning` или `Info` — столбец «Уровень»
`string message`	Текст замечания
`DitaNode? node`	Элемент для перехода двойным щелчком и пути в столбце «Место»; null — без перехода
`string? filePath`	Файл; при проверке проекта передавайте `document.FilePath`

6.3 Форматы публикации: проект, карта, условия

Интерфейс IPublishFormatPlugin и классы ядра, которые нужны для обхода карты: DitaProject, MapTree, MapItem, PublishOptions, PublishFilter, RefResolver, AtomicFile.

6.3.1 IPublishFormatPlugin (DitaStudio.Core.Publishing)

```
public interface IPublishFormatPlugin
{
    string Name { get; }
    string FileExtension { get; }   // без точки, например "md"
    void Publish(DitaProject project, string mapPath, PublishOptions options, string
outputPath);
}
```

Таблица 5. DitaProject (DitaStudio.Core.Project)

Член	Что даёт
`RootPath`, `Name`	Папка и имя проекта
`Files`, `Maps`, `Topics`	Файлы проекта (ProjectFile: FullPath, RelativePath, Kind)
`GetDocument(path)`, `TryGetDocument(path)`	Документ из кэша проекта (открытые вкладки — с несохранёнными правками)
`Catalog`	Каталог элементов проекта (с подключённым DTD): `Catalog.Get(name)` — описание, IsInline, IsTopicType
`ResolveKey(key, scope)`, `Keys`	Определение ключа (KeyDefinition: KeyText, ResolvedPath, Href)
`TextWithKeys(node, scope)`	Текст элемента с подставленными ключами
`DocxLayout`, `CustomCssPath`, `Products`	Настройки проекта: оформление, CSS, список продуктов
`ValidateAll(plugins)`	Проверка проекта, как F7

Таблица 6. MapTree и MapItem

Член	Что даёт
`MapTree.Build(project, mapPath, include)`	Дерево карты (с вложенными картами); include — фильтр условий, обычно `node => PublishFilter.IsIncluded(node, options)`
`Root.Title`	Название публикации
`PublicationOrderWithHeadings`	Порядок издания: топики и разделы без своего топика
`PublicationOrder`	Только топики
`RelatedLinks`	Связи «Смотрите также» из таблиц соответствий
`MapItem.Title`, `Level`	Название строки (с ключами) и глубина: 1 — глава
`TargetPath`, `TargetTopicId`	Файл топика и id вложенного топика (null — корень файла)
`IsHeadingOnly`, `IsPublishedTopic`	Раздел без своего топика / топик, который выходит в публикацию
`KeyScopeChain`	Области ключей строки — для ResolveKey и TextWithKeys
`Node`, `Children`, `Parent`	Элемент карты и дерево строк

Таблица 7. Условия, conref, запись

Член	Что даёт
`PublishOptions`	Условия сборки (ExcludeConditions, IncludeConditions, FlagConditions), ShowDraftComments, Language
`PublishFilter.IsIncluded(node, options)`	Входит ли элемент в публикацию по условиям
`RefResolver.ExpandConrefs(project, document)`	Копия документа с подставленными conref и conkeyref
`RefResolver.FindById(root, id)`	Элемент по id
`AtomicFile.WriteAllText(path, text, encoding)`, `Write(path, stream => …)`, `WriteVia(path, temp => …)`	Надёжная запись: через временный файл (DitaStudio.Core.IO)
`HtmlPublisher`, `DocxPublisher`	Встроенные форматы — можно вызвать из своего плагина, например собрать HTML и упаковать его

6.4 Документ в «Авторе»: IDocumentView

Интерфейс IAuthorCommandPlugin и вкладка документа IDocumentView с операциями режима «Автор» IAuthorSurface (DitaStudio.Presentation).

6.4.1 IAuthorCommandPlugin (DitaStudio.Presentation.Plugins)

```
public interface IAuthorCommandPlugin
{
    string Name { get; }
    void Execute(IDocumentView pane);
}
```

Таблица 8. IDocumentView (DitaStudio.Presentation.Services)

Член	Что даёт
`Document`	Модель документа (DitaDocument) — её и меняет команда
`FilePath`, `Title`, `IsDirty`	Файл, заголовок вкладки, признак несохранённости
`Mode`	Режим вкладки: Author, Source, Preview
`Author`	Операции «Автора» (IAuthorSurface)
`PushUndo(description)`	Снимок для отмены — до правки
`FocusNode(node)`	Показать элемент и поставить в него курсор
`GoToSourceLine(line)`	Перейти к строке исходного кода
`ReloadViews()`	Перестроить все режимы из модели (редактор делает это и сам после команды)
`Save(out error)`	Сохранить документ

Таблица 9. IAuthorSurface: главное

Член	Что даёт
`CurrentNode`	Элемент под курсором
`HasTextSelection`, `HasCellSelection`	Есть выделенный текст / выделены ячейки таблицы
`InsertElement(name)`	Вставить элемент после текущего блока (по контент-модели)
`InsertInlineNode(node)`	Вставить фразовый узел на место курсора
`WrapCurrentInline(name)`, `ToggleCurrentInline(name)`	Обернуть выделение (b, codeph…) / снять оформление
`MoveCurrent(up)`, `DeleteCurrent()`	Переставить и удалить текущий элемент
`ToggleCurrentOutputClass(name)`	Класс в outputclass текущего элемента
`MarkCurrentInserted()`, `MarkCurrentDeleted()`	Пометки правок status
`Rebuild()`	Перестроить «Автор» из модели

6.5 Шаблоны файлов

Файл проекта плагина и заготовки классов всех трёх видов — для копирования.

6.5.1 CompanyRules.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <!-- В вывод — только сборка плагина. -->
    <CopyLocalLockFileAssemblies>false</CopyLocalLockFileAssemblies>
    <!-- Папка установленного редактора: ссылки на его сборки и установка плагина после
сборки. -->
    <DitaStudioDir>C:\Programs\DitaStudio</DitaStudioDir>
  </PropertyGroup>

  <ItemGroup>
    <!-- Private=false: сборки редактора НЕ копируются в вывод плагина. -->
    <Reference Include="DitaStudio.Core">
      <HintPath>$(DitaStudioDir)\DitaStudio.Core.dll</HintPath>
      <Private>false</Private>
    </Reference>
    <!-- Нужна только командам «Автора» (IAuthorCommandPlugin, IDocumentView). -->
    <Reference Include="DitaStudio.Presentation">
      <HintPath>$(DitaStudioDir)\DitaStudio.Presentation.dll</HintPath>
      <Private>false</Private>
    </Reference>
  </ItemGroup>

  <!-- После сборки — копия в plugins/<Имя>/ установленного редактора (перезапустите
его). -->
  <Target Name="InstallPlugin" AfterTargets="Build"
Condition="Exists('$(DitaStudioDir)')">
    <Copy SourceFiles="$(TargetPath)"
DestinationFolder="$(DitaStudioDir)\plugins\$(AssemblyName)" />
  </Target>

</Project>
```

6.5.2 Правило проверки

```
using DitaStudio.Core.Model;
using DitaStudio.Core.Validation;

namespace CompanyRules;

public sealed class NoTodoRule : IValidationRulePlugin
{
    public string Name => "Нет пометок TODO";

    public IEnumerable<ValidationIssue> Check(DitaDocument document)
    {
        foreach (var node in document.Root.DescendantsAndSelf())
        {
            if (node.Kind == NodeKind.Text && node.Value.Contains("TODO",
StringComparison.Ordinal))
            {
                yield return new ValidationIssue(IssueSeverity.Error,
                    "В тексте осталась пометка TODO", node.Parent, document.FilePath);
            }
        }
    }
}
```

6.5.3 Формат публикации

```
using System.Text;
using DitaStudio.Core.IO;
using DitaStudio.Core.Project;
using DitaStudio.Core.Publishing;

namespace CompanyRules;

public sealed class PlainTextFormat : IPublishFormatPlugin
{
    public string Name => "Простой текст";

    public string FileExtension => "txt";

    public void Publish(DitaProject project, string mapPath, PublishOptions options,
string outputPath)
    {
        var tree = MapTree.Build(project, mapPath, node => PublishFilter.IsIncluded(node,
options));
        var sb = new StringBuilder();
        foreach (var item in tree.PublicationOrderWithHeadings)
        {
            sb.AppendLine(item.Title.ToUpperInvariant());
            if (item.TargetPath is not null)
            {
                var doc = RefResolver.ExpandConrefs(project,
project.GetDocument(item.TargetPath));
                sb.AppendLine(doc.Root.FirstElement("shortdesc")?.InnerText.Trim());
            }

            sb.AppendLine();
        }

        AtomicFile.WriteAllText(outputPath, sb.ToString(), new UTF8Encoding(false));
    }
}
```

6.5.4 Команда «Автора»

```
using DitaStudio.Core.Model;
using DitaStudio.Presentation.Plugins;
using DitaStudio.Presentation.Services;

namespace CompanyRules;

public sealed class MarkDraftCommand : IAuthorCommandPlugin
{
    public string Name => "Пометить блок как черновик";

    public void Execute(IDocumentView pane)
    {
        if (pane.Author.CurrentNode is not { } node)
        {
            throw new InvalidOperationException("Поставьте курсор в блок.");
        }

        pane.PushUndo(Name);
        node.SetAttribute("status", "new");
        node.SetAttribute("rev", "draft");
    }
}
```

7 Если плагин не работает

7.1 Плагина нет в списке

Сборка лежит в папке plugins, ошибок загрузки нет, но в окне «Справка → Плагины…» плагина нет.

7.1.1 Признак

Окно «Плагины» не показывает ваш плагин и не называет его сборку среди незагрузившихся.

Причина

Рядом с плагином лежит своя копия `DitaStudio.Core.dll` (или `DitaStudio.Presentation.dll`). Плагин получил свои интерфейсы вместо интерфейсов редактора, и редактор не узнаёт его классы.

Убрать сборки редактора из вывода плагина

Порядок действий

1. **Поставьте у ссылок на сборки редактора `<Private>false</Private>` (или `Private="false"` у `ProjectReference`).**
2. **Удалите `DitaStudio.*.dll` из папки плагина, пересоберите и установите его заново.**

Причина

Класс плагина не подходит: не `public`, абстрактный, вложен в другой непубличный класс или у него нет открытого конструктора без параметров.

Сделать класс открытым

Порядок действий

1. **Объявите класс как `public sealed class … : IValidationRulePlugin` и уберите параметры конструктора (настройки читайте из файла или констант).**

Причина

Сборка лежит в подпапке под другим именем: в `plugins/Rules/` редактор ищет только `Rules.dll`.

Назвать подпапку по сборке

Порядок действий

1. **Переименуйте подпапку по имени сборки или положите сборку прямо в `plugins`.**

Причина

Редактор не перезапускали после установки плагина.

Перезапустить редактор

Порядок действий

1. **Закройте редактор и запустите его снова.**

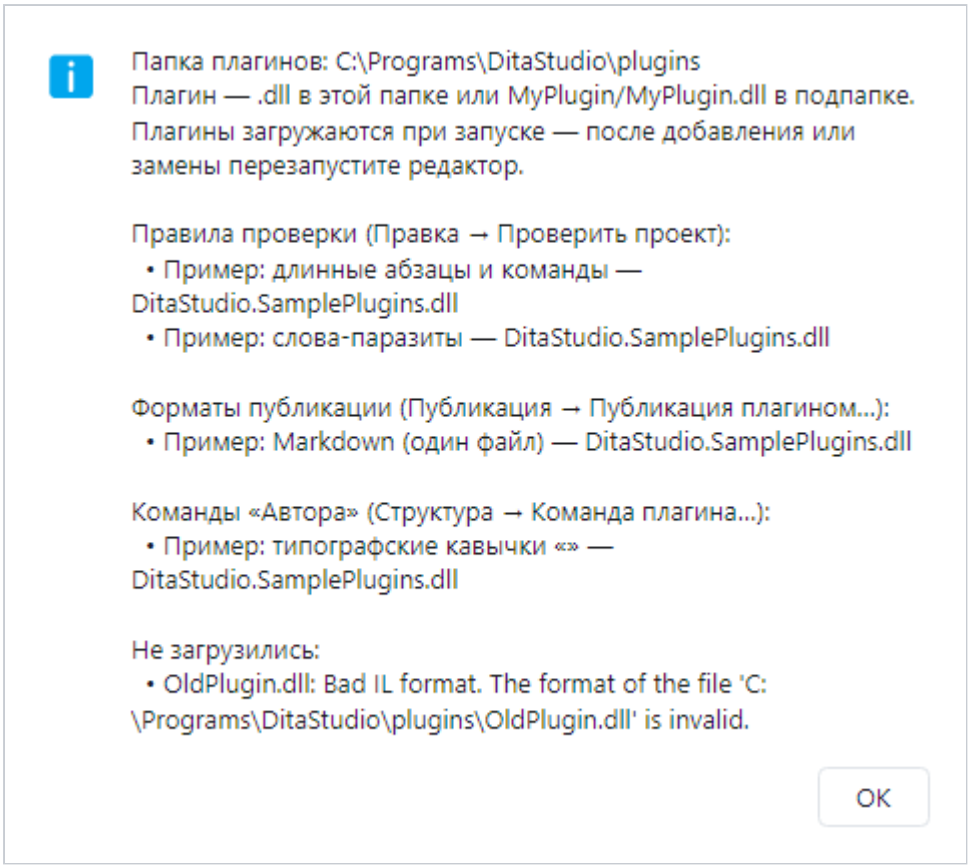
7.2 Сборка не загрузилась

При запуске в строке состояния «Плагины: не загрузилось N», в окне «Справка → Плагины…» — раздел «Не загрузились» с причиной.

7.2.1 Признак

В окне «Плагины» — строка вида «ИмяСборки.dll: причина».

Рисунок 6. Сборка, которая не загрузилась



Причина

`Bad IL format`, `is not a valid .NET assembly` — в папке лежит не сборка .NET: нативная библиотека, обрезанный при копировании файл или `.dll` от другой программы.

Убрать лишний файл

Порядок действий

1. **Удалите файл из `plugins`; нативные зависимости плагина положите в его подпапку — там они плагинами не считаются.**

Причина

`Could not load file or assembly …` — не нашлась зависимость плагина.

Положить зависимости рядом

Порядок действий

1. **Скопируйте весь вывод сборки плагина (вместе с `.deps.json`, кроме сборок редактора) в его подпапку `plugins/<Имя>/`.**

Причина

`Method not found`, `Could not load type`, другая версия `DitaStudio.Core` — плагин собран для другой версии редактора, или целевая платформа плагина не `net8.0`.

Пересобрать для этой версии

Порядок действий

1. **Пересоберите плагин со сборками установленного редактора и `<TargetFramework>net8.0</TargetFramework>`.**

7.3 Плагин падает при работе

Плагин загрузился, но при проверке, публикации или команде появляется «Плагин … упал».

7.3.1 Признак

Правило проверки: на вкладке **Проверка** — предупреждение «Плагин … упал при проверке» с текстом исключения. Формат публикации или команда: окно «Плагин … упал: …».

Причина

Плагин не ожидал такого документа: нет нужного элемента (`FirstElement` вернул null), пустой атрибут, документ — карта, а не топик, или `FilePath` равен null у документа, открытого не из файла.

Найти и исправить место

Порядок действий

1. **Повторите действие с подключённым отладчиком (см. Отладка и тесты) и включите остановку на исключениях.**
2. **Добавьте проверки на null и тест на документ, на котором плагин падал.**